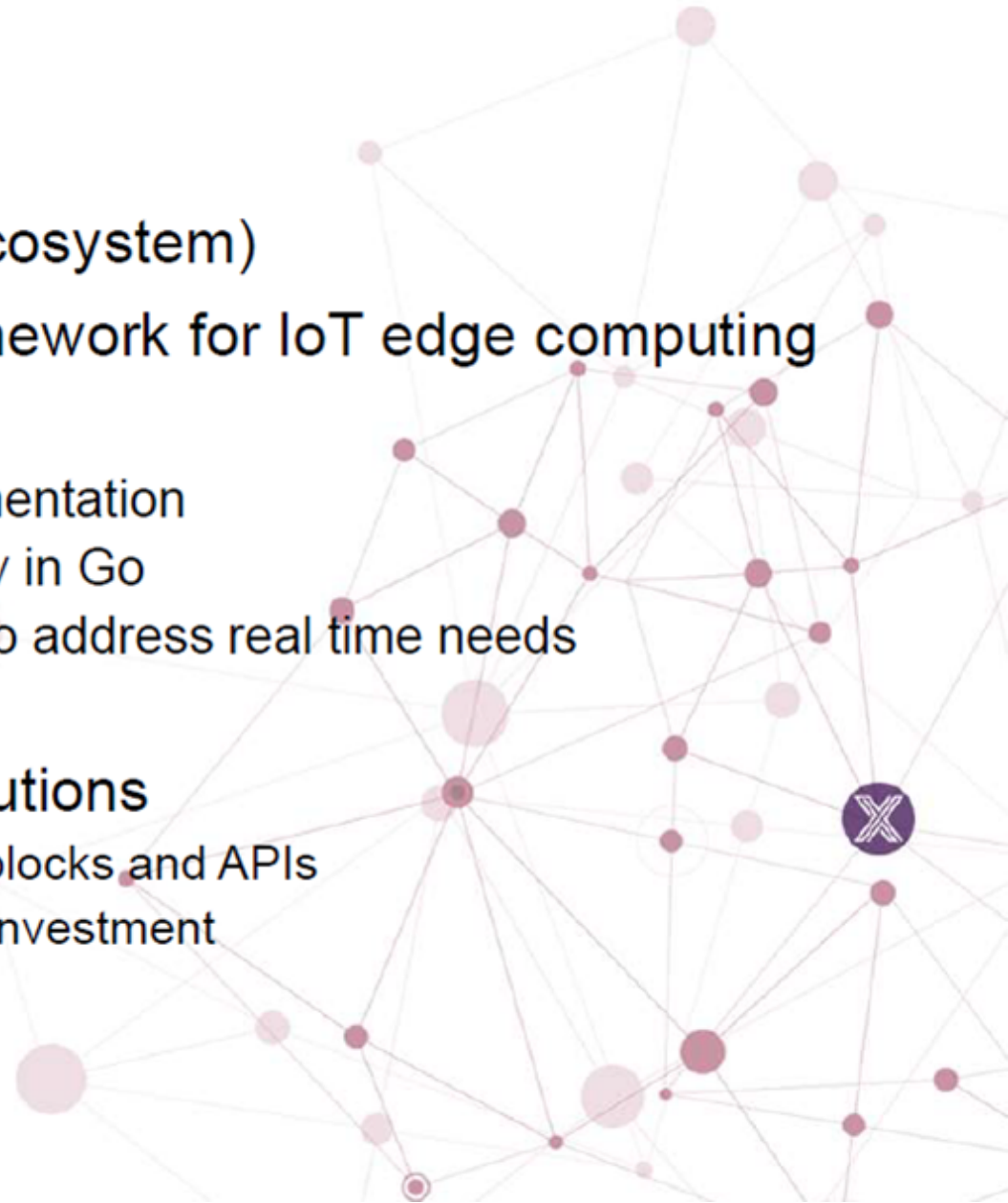


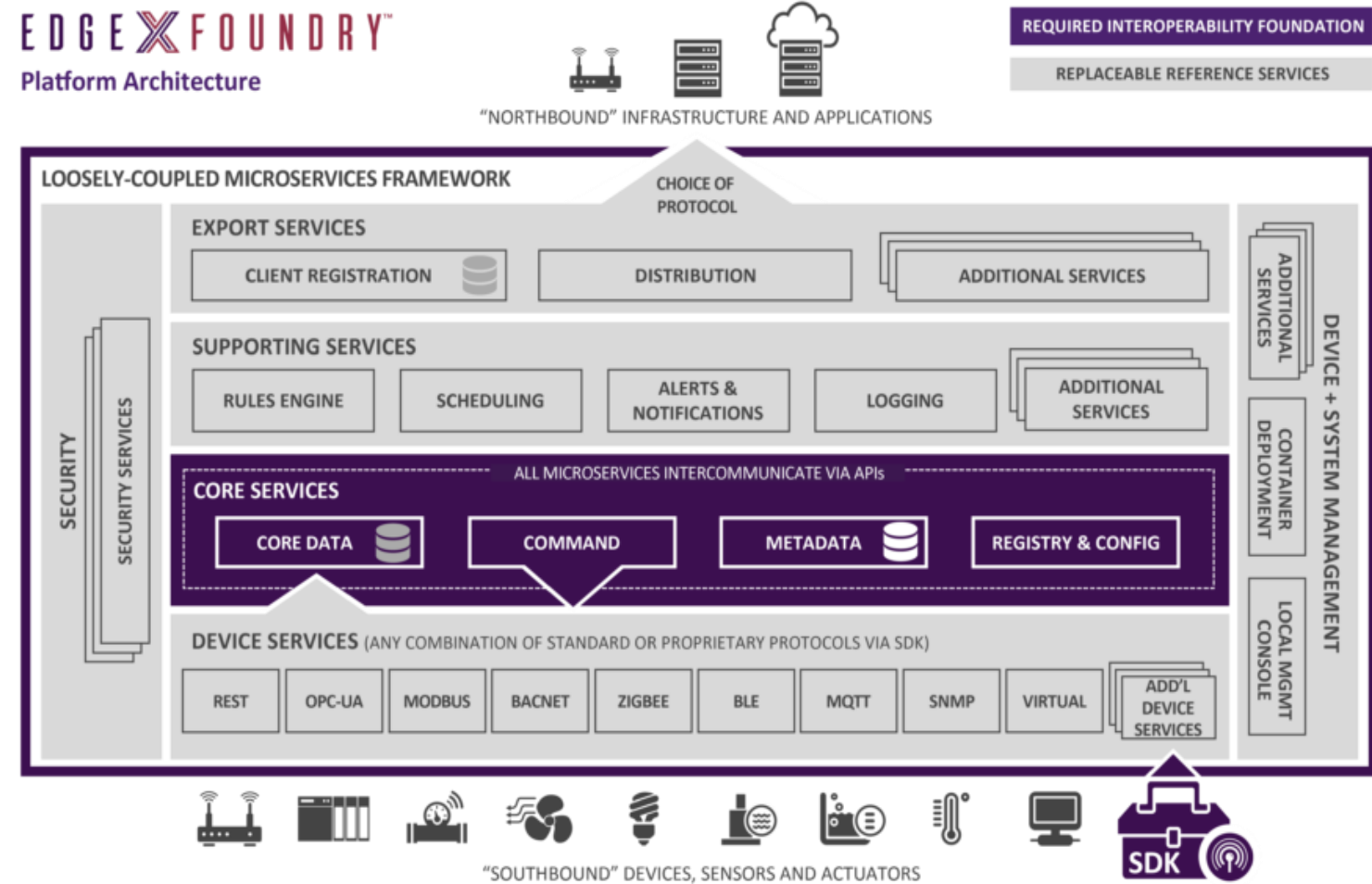
Introducing EdgeX Foundry

- An open source, vendor neutral project (and ecosystem)
- A **microservice**, loosely coupled software framework for IoT edge computing
- Hardware and OS agnostic
 - Remain agnostic with regard to microservice implementation
 - Many of the microservices were in Java and are now in Go
 - C/C++ is envisioned for south side connectors and to address real time needs
 - JavaScript for UI
- Goal: enable and encourage growth in IoT solutions
 - The community builds and maintains common building blocks and APIs
 - Plenty of room for adding value and getting a return on investment
 - Allowing best-of-breed solutions



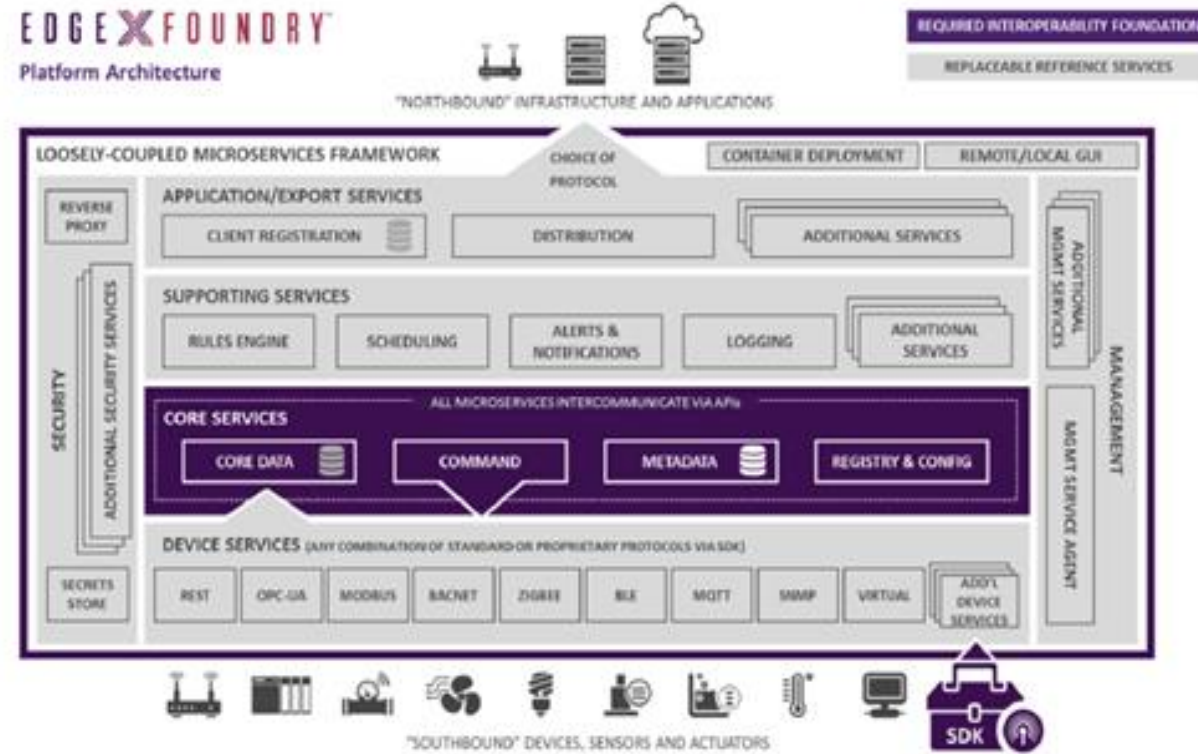
EdgeX Micro Service Architecture

- Contextually, EdgeX micro services are divided into 4 layers
- Device Services (device services for various protocols like Modbus, BACnet, ... and SDK to create new device services)
 - Communicate in native sensor/device protocol to the physical – that is to the IoT “things”
 - Transform sensor data to common format
 - Translate command requests to actuate devices (in native protocol/format)
- Core Services (core data, metadata, command & configuration/registration)
 - Offers temporary persistence of edge data and facilitates actuation of things through common API
 - Collect sensor data
 - Understand what sensors/devices are connected how to communicate with them (metadata)
 - Provision facility for new sensors/devices (and device services)
 - Manage device actuation requests to device services/devices
 - Provide micro service registry
 - Provide micro service configuration
- Supporting Services (logging, notifications, scheduler, rules engine)
 - Normal software application duties plus “edge intelligence”
 - Logging
 - Notifications and alerting
 - Scheduling and clean up
 - Rules engine
- Export Services (client, distribution)
 - On or off box client registration of data
 - Distribution center of sensor data to clients
 - Transport edge data to the enterprise and cloud systems in a manner they request



It's All About the APIs

- **Loosely-coupled microservices** bound by common APIs established through vendor-neutral collaboration in Linux Foundation
- **HW-, OS- and Protocol-agnostic**
- Polyglot: microservices can be written in any programming language (e.g. Java, Python, Go Lang, C) and deployed in containers or VMs
- **Granularity in API definition facilitates bringing together heterogeneous OSS and Commercial offerings for various functions**
- Once key APIs are established, **entire subsections can be replaced, combined, etc. with proprietary, differentiated “EdgeX-compliant” offerings**, even Core Services



EdgeX is architected to enable **commercial value-add** around a lowest common denominator interoperability framework.

EdgeX Technology

- A majority of the micro services are written in Go Lang
 - Previously written in Java
 - Some Device Services written in C/C++
 - A user interface is provided in JavaScript
 - Polyglot belief – use the language and tools that make sense for each service
- Each service has a REST API for communicating with it
- Uses MongoDB to persist sensor data at the edge
 - Also stores application relevant information
 - Allows for alternate persistence storage (and has been done in the past)
- A message pipe connects Core Data to Export Services and/or Rules Engine
 - Uses ZeroMQ by default
 - Allow use of MQTT as alternate if broker is provided
- Uses open source technology where appropriate
 - Ex: Consul for configuration/registry, Kong for reverse proxy, Drools for rules engine,...



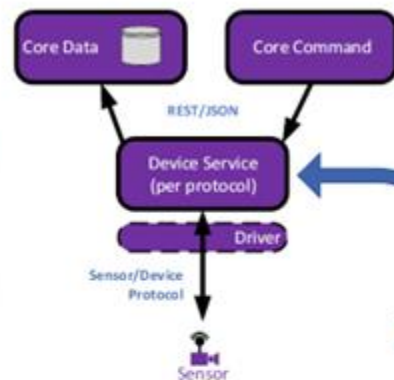
{ REST }



Transformation on the “South Side”

- The world of OT protocol soup
 - Modbus, BACNet, Profibus, CANBus, OPC-UA...
- Consumer and traditional IT protocols are also entering the mix
 - BLE, Zigbee, ZWave, MQTT, SNMP, ...
- New “things” & protocols are being added all the time
 - You can never keep up with them all
- EdgeX Device Services transform from “thing” protocols and data to common Core Data (micro) Service

Device Services



Transformation on the “North Side”

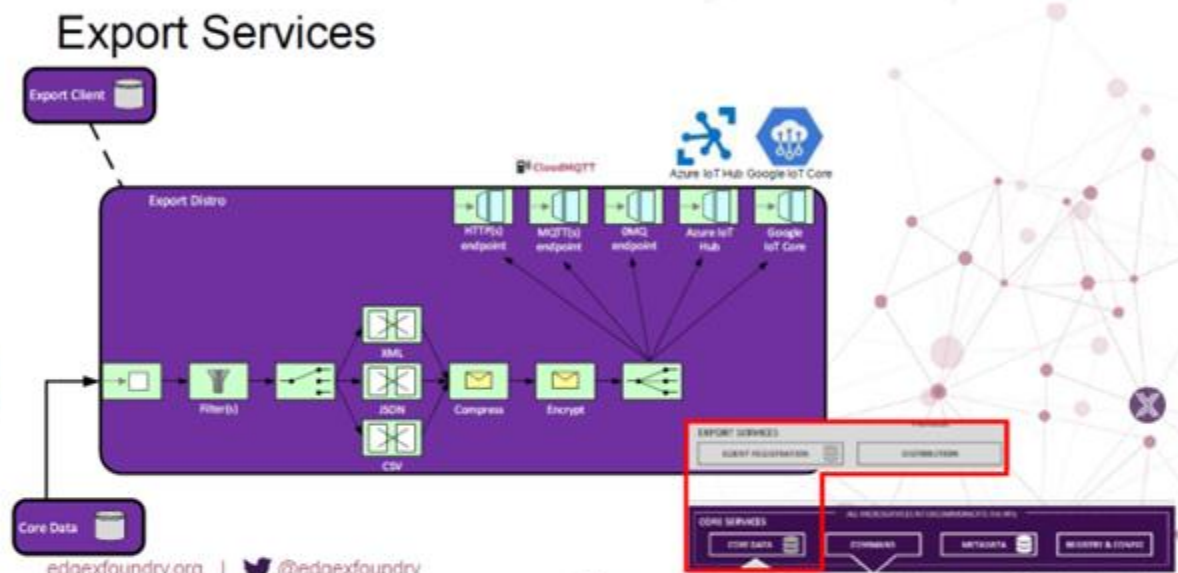
- North side endpoints need data...

- Filtered
- Transformed (data model of choice)
- Enriched (add device metadata, location, etc.)
- Formatted (XML, JSON, CSV, ...)
- Compressed, Encrypted, etc.

- This is classic EAI (enterprise application integration)

- aka pipe & filter architecture

- EdgeX Export Services take the data from Core Data and get it to the applications/cloud

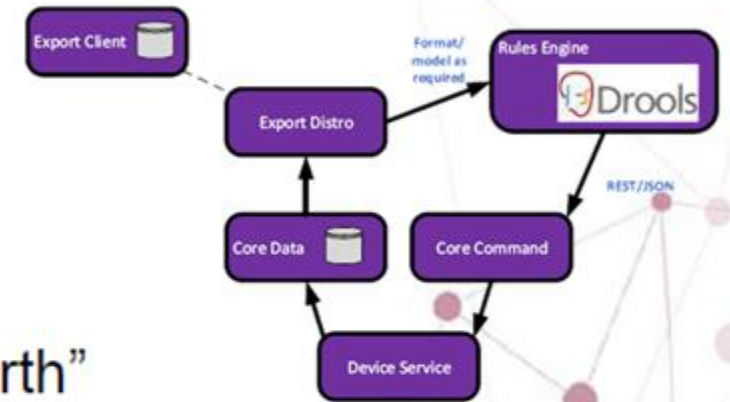


edge analytics

- Intelligence or analytics at the edge is critical
 - It's too expensive to transport all the data "north"
 - It's too expensive to store all the data "north"
 - It's too late to react to a problem from the cloud
 - Your devices/sensors are not always connected to the "north"
- How smart does your edge platform need to be?
 - Simple rule engine smart?
 - Complex event process (CEP) smart?
 - Machine learning/AI smart?
- The edge platform must be flexible enough to incorporate different capability
- EdgeX's analytic service can wrap and isolate the edge analytic capability

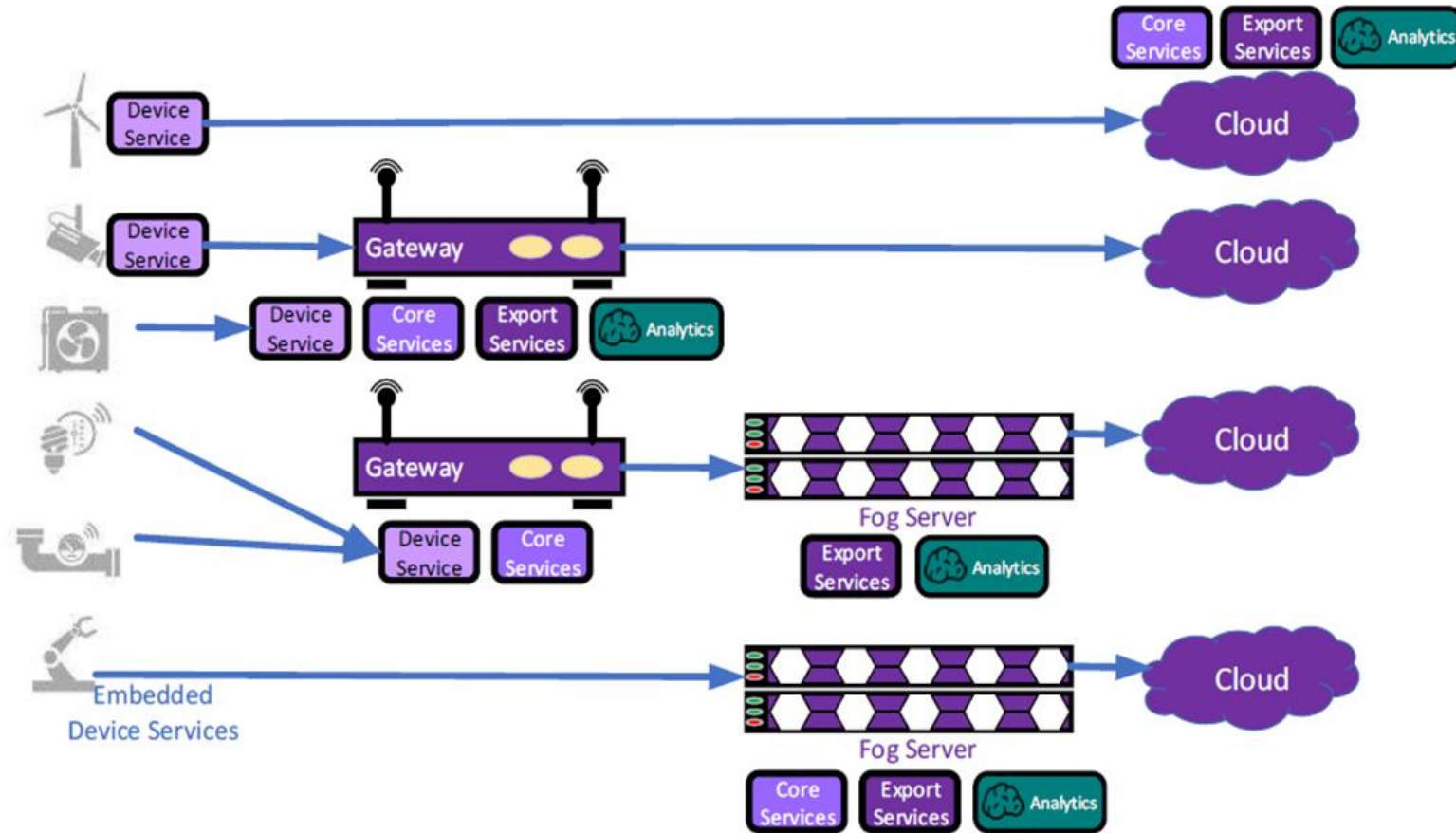
Rules Engine Service

EdgeX Reference Implementation

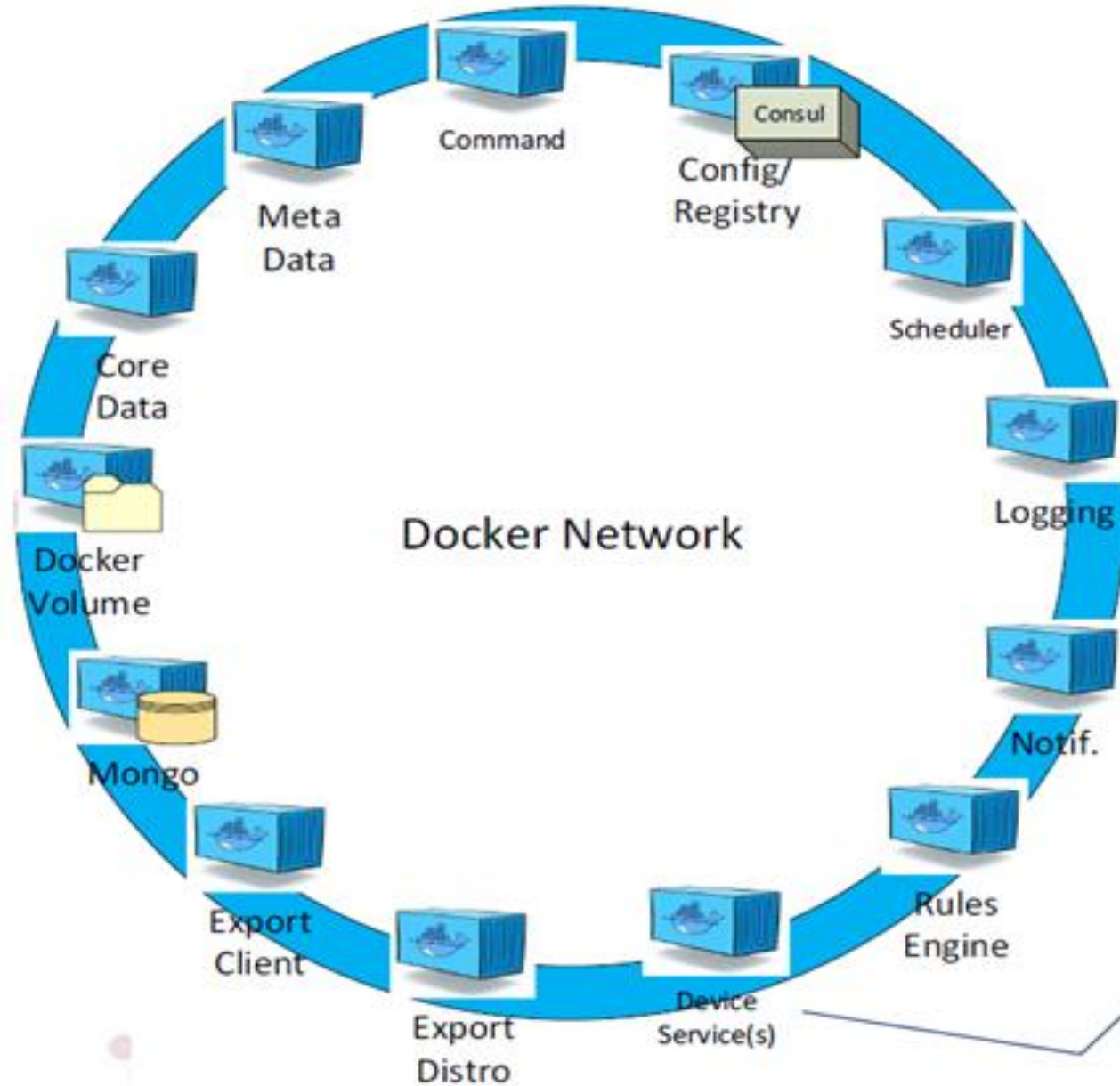


EdgeX Flexible Deployment Possibilities

- In today's IoT landscape, it is imperative to leverage compute, storage, network resources where ever they live
- Loosely-coupled architecture enables distribution across nodes to enable tiered edge/fog computing
- Scope includes embedded sensors to controllers, edge gateways and servers
- Quantity and function of micro services deployed on a given node depends on th use case and capability of hardware



The Typical EdgeX Deployment



Number of device services will vary per use case and number of sensors/devices

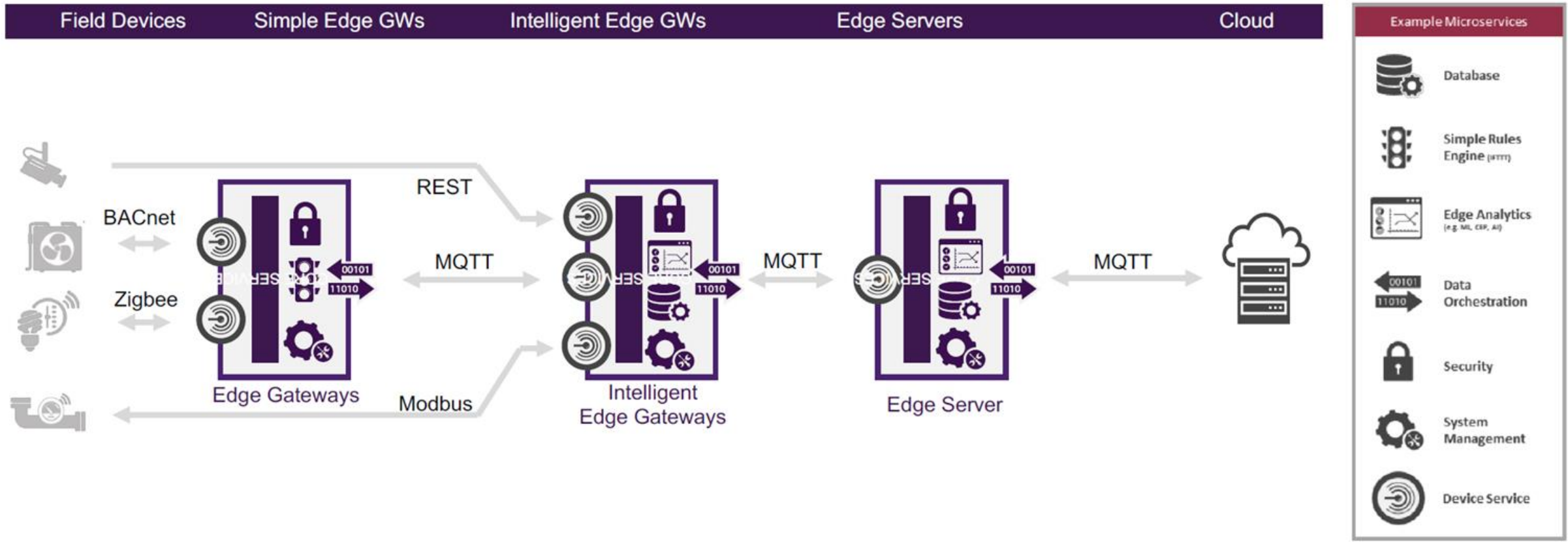
Bridging Standards with An Ecosystem of Applications



The EdgeX framework enables developers to **decouple their preferred Edges and Clouds** as close to the physical world as possible, **minimizing reinvention of table stakes elements** for data ingestion, security and management and benefitting from **flexibility and choice** (e.g. build vs. buy from the open ecosystem).

Tiered Edge Deployment

Number of deployed microservices and functionality increases higher in tier



The EdgeX Microservices

Core Services - Where most of the knowledge as to what “things” are connected, what data is flowing and how EdgeX is configured in a given deployment

Microservice	EdgeX Deployment Name	Description
Core Data	edgex-core-data	Data store for readings collected by devices and sensors. Works in conjunction with the Database service
Core Metadata	edgex-core-metadata	Used by other services for knowledge about the devices and how to communicate with them
Core Command	edgex-core-command	Enables commands to devices on behalf of other services, applications or external systems. Optional
Database	edgex-redis	The default database for EdgeX, implemented via Redis
Configuration & Registry	edgex-core-consul	Centralized service configuration for EdgeX implemented via Consul. Optional

Device Services - The connectors that interact directly with the edge sensors and devices. An EdgeX use case would typically run at least one Device Service. This report provides statistics for the REST and the Virtual Device Service but many more Device Services are provided both by the Foundry and the commercial EdgeX ecosystem.

Microservice	EdgeX Deployment Name	Description
REST Device Service	edgex-device-rest	Device Service for interacting with edge devices that provide a REST-based API
Virtual Device Service	edgex-device-virtual	Device Service for simulating device data

Supporting Services - Includes edge rules, analytics and typical software application duties such as logging, scheduling and data clean up.

Microservice	EdgeX Deployment Name	Description
Rules Engine	edgex-core-data	Data store for readings collected by devices and sensors. Works in conjunction with the Database service
Support Notifications	edgex-core-metadata	Used by other services for knowledge about the devices and how to communicate with them
Support Scheduler	edgex-core-command	Enables commands to devices on behalf of other services, applications or external systems. Optional

Application Services - The means to extract, process and send EdgeX data to an endpoint or application of your choice. A full SDK is provided, but for convenience, EdgeX provides a base application service that can be configured to execute a number of built-in functions and deliver data to northbound endpoints.

Microservice	EdgeX Deployment Name	Description
App Service Configurable	edgex-app-service-configurable-rules	Application Service that can be configured to execute built-in transform and export functions

EdgeX Footprint and Container Sizes

Each EdgeX microservice is typically implemented in either Go or C and then compiled into an executable which has a size or footprint as it sits on disk. For convenient deployment and orchestration, each microservice executable can also be built into a container image so it can be run as a standalone service with less dependencies on the host. This greatly improves portability and platform independence. A container image will be larger than the base executable because it may also contain a minimal operating system and any required configuration and supporting infrastructure. EdgeX provides container images for both Docker and Ubuntu Snaps.

Both containerized Docker image footprint data and non-containerized executable footprint data is measured.

Typical EdgeX Deployment

EdgeX Microservice	Image Footprint (MB)	Executable Footprint (MB)
Core Services		
edgex-core-data	23.69	15.48
edgex-core-metadata	14.49	14.48
edgex-core-command	12.17	12.17
edgex-redis	31.15	N/A
edgex-core-consul	141.05	N/A
Device Services		
edgex-device-virtual	20.48	14.85
edgex-device-rest	17.09	11.50
Supporting Services		
edgex-support-notifications	13.63	12.30
edgex-support-scheduler	12.30	12.29
edgex-kuiper	20.24	N/A
Application Services		
edgex-app-service-configurable-rules	28.94	20.21
TOTAL	335.23	

Minimal EdgeX Deployment

EdgeX Microservice	Image Footprint (MB)	Executable Footprint (MB)
Core Services		
edgex-core-data	23.69	15.48
edgex-core-metadata	14.49	14.48
edgex-redis	31.15	N/A
Device Services		
edgex-device-rest	17.09	11.50
Application Services		
edgex-app-service-configurable-rules	28.94	20.21
TOTAL	127.53	

The Docker-based image footprint for a typical EdgeX deployment is approximately 335 MB and approximately 128 MB for a minimal deployment. The optional Configuration & Registry service (Consul) is the most significant contributor to the footprint.

Note: The executable footprint is not recorded for third-party open source services (Redis, Consul, Kuiper, etc) so N/A is listed for those services.

EdgeX CPU Consumption

Each EdgeX microservice has its CPU consumption measured as it is started as a Docker container. The CPU usage is reported by the Docker engine and is measured as a percentage of the available CPU on the machine. In general, the measure of usage at startup is a good indication on the upper bound for many of the services. Note that the characteristics of different chip architectures may have a significant effect on the percentage of CPU that is utilized.

Typical Deployment Memory Usage

EdgeX Microservice	Average (%)	Maximum (%)	Minimum (%)
Core Services			
edgex-core-data	1.51	1.56	1.44
edgex-core-metadata	1.56	1.68	1.39
edgex-core-command	1.44	1.49	1.35
edgex-redis	0.21	0.36	0.18
edgex-core-consul	1.36	1.87	1.12
Device Services			
edgex-device-virtual	0.08	0.11	0.00
edgex-device-rest	0.00	0.04	0.00
Supporting Services			
edgex-support-notifications	1.46	1.58	1.27
edgex-support-scheduler	1.42	1.54	1.27
edgex-kuiper	0.00	0.00	0.00
Application Services			
edgex-app-service-configurable-rules	0.00	0.00	0.00
TOTAL	9.04	10.23	8.02

Minimal EdgeX Deployment

EdgeX Microservice	Average (%)	Maximum (%)	Minimum (%)
Core Services			
edgex-core-data	1.51	1.56	1.44
edgex-core-metadata	1.56	1.68	1.39
edgex-redis	0.21	0.36	0.18
Device Services			
edgex-device-rest	0.00	0.04	0.00
Application Services			
edgex-app-service-configurable-rules	0.00	0.00	0.00
TOTAL	3.28	3.64	3.01

CPU usage recorded on the test hardware for a typical EdgeX deployment is approximately 10% and approximately 3% for a minimal deployment.

EdgeX Memory Consumption

Each EdgeX microservice has its memory consumption measured as it is started as a Docker container. The memory consumption is reported by the Docker engine and is measured in Megabytes (MB). In general, the measure of usage at startup is a good indication on the upper bound for many of the services.

Typical EdgeX Deployment

EdgeX Microservice	Average (MB)	Maximum (MB)	Minimum (MB)
Core Services			
edgex-core-data	12.92	13.61	12.30
edgex-core-metadata	13.15	13.53	12.91
edgex-core-command	9.20	9.99	8.08
edgex-redis	5.32	5.82	5.03
edgex-core-consul	48.89	49.09	48.67
Device Services			
edgex-device-virtual	13.96	14.41	13.44
edgex-device-rest	11.72	11.80	11.63
Supporting Services			
edgex-support-notifications	9.55	10.08	8.74
edgex-support-scheduler	9.91	10.35	9.24
edgex-kuiper	21.61	21.61	21.61
Application Services			
edgex-app-service-configurable-rules	11.86	12.08	11.26
TOTAL	168.09	172.34	162.90

Minimal EdgeX Deployment

EdgeX Microservice	Average (MB)	Maximum (MB)	Minimum (MB)
Core Services			
edgex-core-data	12.92	13.61	12.30
edgex-core-metadata	13.15	13.53	12.91
edgex-redis	5.32	5.82	5.03
Device Services			
edgex-device-rest	11.72	11.80	11.63
Application Services			
edgex-app-service-configurable-rules	11.86	12.08	11.26
TOTAL	54.97	56.84	53.13

Memory usage recorded on the test hardware for a typical EdgeX deployment is approximately 170 MBs and approximately 56 MB for a minimal deployment.

EdgeX Startup Times

The startup times are measured for each of the microservices developed by the EdgeX Foundry community.

Startup times include any overhead associated with creating the Docker container in which the microservices runs. Note that the metrics are obtained while starting all microservices at the same time so any dependencies between the services starting is included in the data. The total time recorded is the time taken for all of the services to be started, rather than a sum of all individual startup times.

Note also that services can be started and stopped as EdgeX runs. In this case it is not necessary to create a new Docker container for each service. Restarting an already-created Docker container reduces the startup overhead slightly

EdgeX Startup Times

EdgeX Microservice	Average (s)	Maximum (s)	Minimum (s)
Core Services			
edgex-core-data	9.10	11.34	7.82
edgex-core-metadata	9.15	11.19	8.03
edgex-core-command	9.10	11.15	7.69
Device Services			
edgex-device-virtual	17.64	18.51	16.46
edgex-device-rest	17.40	18.35	16.13
Supporting Services			
edgex-support-notifications	9.09	11.23	8.00
edgex-support-scheduler	9.17	11.29	8.06
Application Services			
edgex-app-service-configurable-rules	9.37	11.47	8.27
TOTAL	17.64	18.51	16.46

The startup time for the EdgeX microservices on the test hardware is approximately 18 seconds.

EdgeX Operational Latency

Ping and Data Export Latency

The ping responses times are measured for each of the microservices developed by the EdgeX Foundry community and represent the reactivity of each service when it receives an HTTP/REST request

Ping Response Times

EdgeX Microservice	Average (ms)	Maximum (ms)	Minimum (ms)
Core Services			
edgex-core-data	0.98	4.35	0.79
edgex-core-metadata	0.98	2.03	0.79
edgex-core-command	1.16	1.74	0.78
Device Services			
edgex-device-virtual	1.12	1.57	0.88
edgex-device-rest	1.12	1.52	0.87
Supporting Services			
edgex-support-notifications	1.14	1.59	0.92
edgex-support-scheduler	1.17	1.83	0.93
Application Services			
edgex-app-service-configurable-rules	1.23	6.03	0.93

Ping response times recorded on the test hardware are consistently around 1ms for each of the EdgeX microservices.

Data export latency represents the time it takes to collect EdgeX data from the "southside" devices & sensors and deliver it through the platform to the "northside" applications.

The measurement represents the time taken for an event reading to be created in a Device Service, delivered to the Core Data Service and then passed to an Application Service where it is read and prepared for northbound delivery. The event is extracted from the Core Data event message topic, sent to the designated endpoint and finally marked as 'pushed' in Core Data.

The latency reported here relates to the Virtual Device Service, which simulates 3 different data attributes. Each Device Service type (Modbus, BACnet, etc), will obviously have its own data rates associated with collecting the data from the physical edge devices or sensors, but the Virtual Device provides useful indicative performance data. It is planned to provide performance relating to more Device Service types in future versions.

Data Export Latency

Virtual Device Name	Average (ms)	Maximum (ms)	Minimum (ms)
Core Services			
Random-Integer-Device	3.5	7.0	2.0
Random-UnsignedInteger-Device	4.8	8.0	3.0
Random-Boolean-Device	6.0	9.0	4.0

The Virtual Device data is delivered through the EdgeX platform on the test hardware at an average latency of approximately 5 ms.

EdgeX Foundry



Brief Description

EdgeX, your data liberated! Highly flexible open source software framework that facilitates interoperability between heterogeneous devices and applications at the IoT Edge, along with a consistent foundation for security and manageability regardless of use case.

Contributed by

Dell in April 2017

Key Contacts

Jim White, IOTech, TSC Chair

Henry Lau, HP Inc., TAC Voting Member

Key Links

Website <https://www.lfedge.org/projects/edgexfoundry/>

Wiki <https://wiki.edgexfoundry.org/>

GitHub <https://github.com/edgexfoundry>

Documentation <https://docs.edgexfoundry.org/>

Mail Lists <https://lists.edgexfoundry.org/g/main/subgroups>

Slack <https://slack.edgexfoundry.org/>

Technical Steering Committee (TSC)

<https://wiki.edgexfoundry.org/pages/viewpage.action?pageId=329436>

Getting Started Guide

<https://docs.edgexfoundry.org/1.2/getting-started/>

eKuiper



Brief Description

eKuiper is an edge lightweight IoT data analytics / streaming software implemented by Golang, and it can be run at all kinds of resource constrained edge devices.

Contributed by

EMQ in June 2021

Key Contacts

Jiyong Huang, EMQ, TSC Representative

Key Links

Website <https://www.lfedge.org/projects/>

Wiki <https://wiki.lfedge.org/display/EKUIPER/eKuiper>

GitHub <https://github.com/lf-edge/ekuiper>

Mail Lists <https://lists.lfedge.org/g/ekuiper>

<https://lists.lfedge.org/g/ekuiper-tsc>

Slack <https://slack.lfedge.org/>

(#ekuiper / #ekuiper-tsc)

Technical Steering Committee (TSC)

<https://wiki.lfedge.org/display/EKUIPER/%28TSC%29+Technical+Steering+Committee>

Quick Start

<https://wiki.lfedge.org/display/EKUIPER/eKuiper+5+minutes+quick+start>

Requirements for Enabling Edge Computing

